

WMCS045-05 – Lab Assignment #4

Jort Oosterveld

2026-09-30

1 Squared Sums

1.1 System

All measurements were taken on the system in Table 1. The benchmark was pinned to one core with `taskset`.

Table 1: System used for the measurements.

CPU	AMD Ryzen 7 5825U (Zen 3, family 25, model 80)
Cores / threads	8 / 16
Frequency	max. 4.55 GHz (boost enabled), 4.50 GHz assumed for cycle counts
Caches	L1d 32 KiB and L2 512 KiB per core, L3 16 MiB shared
SIMD registers	16 (AVX2, FMA3)
OS	Linux 7.0.0-31-generic
Governor	<code>performance</code> (<code>amd-pstate-epp</code>)
Compiler	<code>g++</code> (GCC) 15.2.1
Flags	<code>-std=c++17 -O2 -march=native</code> <code>-fno-tree-vectorize -ffp-contract=off</code>

The microarchitecture parameters of Zen 3 that matter for this exercise are listed in Table 2. They are taken from the AMD Software Optimization Guide for family 19h processors and the instruction tables of uops.info.

Table 2: Relevant microarchitecture parameters of Zen 3 (scalar double precision).

Instruction	Latency (cycles)	Throughput (per cycle)	Execution units
<code>vaddsd</code>	3	2	FP2, FP3
<code>vmulsd</code>	3	2	FP0, FP1
<code>vfmadd231sd</code>	4	2	FP0, FP1
Dispatch width: 6 macro-ops per cycle			

1.2 Benchmark

The loop of `sqsum` is unrolled by a factor U with one accumulator per unrolled statement, which breaks the dependency for the one accumulator. The complete benchmark program `loop_unrolling.cpp` can be found in Listing 3 in Appendix A.

Two compiler flags are needed to measure what is intended. `-fno-tree-vectorize` keeps the loop scalar, and `-ffp-contract=off` keeps the separate multiply and add, otherwise GCC fuses them into an FMA by itself.

As in the assignment, $n = 8192$ and `sqsum` is called 10000 times. This batch of 10000 calls is timed with `clock_gettime(CLOCK_MONOTONIC_RAW)` and repeated 30 times per unrolling

factor. Only the minimum (best case) of the 30 repetitions is reported, since disturbances by the operating system can only make a run slower.

Hardware cycle counters are not accessible on this system (`perf_event Paranoid = 4`). The time per element is therefore converted to cycles with an assumed constant core frequency of 4.50 GHz, the boost clock the core runs at with the `performance` governor while the benchmark is pinned to it. The cycle counts are therefore only as accurate as this assumption, the times are what was measured.

1.3 Results

Table 3 shows the results (`loop_unrolling.csv`).

Table 3: Best case of 30 repetitions for the squared sum with unrolling factor U , $n = 8192, 10000$ calls per repetition. The speed-up is relative to the loop without unrolling.

U	μs / call	ns / element	cycles / element	speed-up
1	5.49	0.6705	3.017	1.00
2	2.78	0.3388	1.525	1.98
3	1.88	0.2297	1.034	2.92
4	1.55	0.1888	0.850	3.55
5	1.25	0.1532	0.689	4.38
6	1.23	0.1498	0.674	4.48
7	1.20	0.1464	0.659	4.58
8	1.07	0.1306	0.587	5.13
9	1.14	0.1390	0.625	4.82
10	1.12	0.1365	0.614	4.91
11	1.06	0.1300	0.585	5.16
12	1.15	0.1404	0.632	4.78
14	1.05	0.1280	0.576	5.24
16	1.62	0.1982	0.892	3.38

1.4 a) Optimal unrolling factor

Without unrolling, every addition has to wait for the previous one, so one element costs the latency of the loop-carried operation. Only the addition is part of this dependency chain, the multiplication depends on the loaded value alone. The measured 3.02 cycles per element for $U = 1$ match the latency of 3 cycles of `vaddsd`.

With U accumulators there are U independent chains, so the cost drops to $3/U$ cycles per element until the throughput of the core becomes the limit. Zen 3 executes 2 additions and 2 multiplications per cycle, which is 0.5 cycles per element. The chains stop being the bottleneck at

$$U_{\text{opt}} = \text{latency} \times \text{throughput} = 3 \times 2 = 6.$$

The measurements follow $3/U$ up to $U = 3$ and then flatten out. after $U = 6$, there is no systematic gain anymore. All factors from 6 to 14 lie between 0.58 and 0.67 cycles per element. This agrees with the expected factor of 6, which is 4.5 times faster than the original loop. The theoretical 0.5 cycles per element is not reached. A likely reason is the dispatch width: the loop needs 3 macro-ops per element, a load, multiply and add. So 0.5 cycles per element would already use all 6 macro-ops per cycle that Zen 3 can dispatch, and the loop overhead comes on top. This also explains why $U = 8$ is clearly faster than $U = 6$ with 0.587 cycles per element. There is less loop overhead (shown in listings 1 and 2 when unrolling is done with a power of two, which gives an estimate of $(3 \cdot 8 + 2)/(6 \cdot 8) = 0.54$ cycles per element against $(3 \cdot 6 + 4)/(6 \cdot 6) = 0.61$ for

$U = 6$. With the same estimate the overhead of 4 macro-ops is spread over more elements for larger factors, so $U = 11$ and $U = 14$ are as fast as $U = 8$. The differences between the factors 8 to 14 can be attributed to variations between runs.

Listing 1: Loop overhead of compiled assembler with $u=6$

```

1    leal    11(%rcx), %edx
2    addq    $48, %rax
3    addl    $6, %ecx
4    cmpl    %edx, %esi
5    jg      .L51

```

Listing 2: Loop overhead of compiled assembler with $u=8$

```

1    addq    $64, %rax
2    cmpq    %rax, %rdx
3    jne     .L80

```

1.5 b) Fused-multiply-add

With FMA, the multiplication and addition are one instruction, so there are fewer 2 macro-ops per element instead of 3. The drawback is that the whole 4 cycles of FMA are dependent on each other, instead of the 3 from the addition. Without unrolling, FMA is therefore expected to be about $\frac{1}{3}$ slower.

The throughput of FMA on Zen 3 is 2 per cycle, the same as for the separate multiply and add. But, more accumulators are needed to reach it, $4 \times 2 = 8$ instead of 6. For every unrolling factor below 8 the dependency chain of the FMA is still the bottleneck, so FMA is slower.

FMA can only be better when the loop is unrolled by at least 8. The execution units then allow the same 0.5 cycles per element for both variants, but the multiply and add loop needs all 6 macro ops per cycle of the dispatch width to get there, which is the likely reason that it only reached 0.58 cycles per element in the measurements. With 2 macro ops per element FMA needs only 4 per cycle, so it could come closer to the limit.

The relevant microarchitecture parameters are:

- the latency of the addition compared to the latency of the FMA, since this is the length of the loop-carried dependency chain,
- the throughput (number of execution units) of addition, multiplication and FMA, which sets the limit once the dependency is broken,
- the pipeline (dispatch) width in relation to the number of macro-ops per element,
- the number of registers, which limits the number of accumulators (15 on this system).

A Benchmark source code

Listing 3 is the complete benchmark program `loop_unrolling.cpp` used for question 1. It is compiled and run with `run_benchmark.sh`, which pins it to one core with `taskset` and writes `loop_unrolling.csv`.

Listing 3: Complete benchmark program `loop_unrolling.cpp`.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>

```

```

4
5 // timer helper
6 long long now_ns()
7 {
8     struct timespec ts;
9     clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
10    return ts.tv_sec * 1000000000LL + ts.tv_nsec;
11 }
12
13 // Squared sum, unrolled by a factor U with one accumulator per unroll
14 template <int U>
15 __attribute__((noinline)) // ask the compiler not to perform inlining to
16    keep the unroll factor the only change
17 double sqsum(double *__restrict a, int n)
18 {
19     double s[U] = {};
20     int i = 0;
21
22     for (; i + U - 1 < n; i += U) {
23         // unroll inner loop fully, because we want all U accumulators
24         // written here
25         #pragma GCC unroll 16 // max unroll we test
26         for (int k = 0; k < U; k++) {
27             s[k] += a[i + k] * a[i + k];
28         }
29     }
30
31     // remainder when n is not a multiple of U
32     for (; i < n; i++) {
33         s[0] += a[i] * a[i];
34     }
35
36     double total = 0.0;
37     for (int k = 0; k < U; k++){
38         total += s[k];
39     }
40
41     return total;
42 }
43
44 struct Kernel {
45     int unroll;
46     double (*fn)(double *__restrict, int);
47 };
48
49 #define KERNEL(U) {U, sqsum<U>},
50
51 // called through this table, so the compiler cannot inline the kernels
52 // or
53 // hoist the calls out of the timing loop
54 const Kernel kernels[] = {
55     KERNEL(1) KERNEL(2) KERNEL(3) KERNEL(4) KERNEL(5) KERNEL(6) KERNEL
56     (7)
57     KERNEL(8) KERNEL(9) KERNEL(10) KERNEL(11) KERNEL(12) KERNEL(14)
58     KERNEL(16)
59 };
60
61 double sum;

```

```

57
58 int main()
59 {
60     int n = 8192;
61     int calls = 10000;
62     int reps = 30;
63
64     // Multiples of 0.25 so no floating point inaccuracies
65     double *a = (double *) calloc(n, sizeof(double));
66     double expected = 0.0;
67     for (int i = 0; i < n; i++) {
68         a[i] = (i % 16) * 0.25;
69         expected += a[i] * a[i];
70     }
71
72     printf("unroll,min_ns,ns_per_element,cycles_per_element\n");
73
74     for (const Kernel& kernel : kernels) {
75         // assume max clock speed for Jort's system
76         double core_ghz = 4.50;
77
78         // best case of 'reps' batches of 'calls' calls
79         long long min_ns = 0;
80         for (int rep = 0; rep < reps; rep++) {
81             long long start = now_ns();
82             for (int i = 0; i < calls; i++) {
83                 sum = kernel.fn(a, n);
84             }
85             long long elapsed = now_ns() - start;
86
87             if (rep == 0 || elapsed < min_ns) {
88                 min_ns = elapsed;
89             }
90         }
91
92         if (sum != expected) {
93             fprintf(stderr, "Error: U=%d returned %f, expected %f\n",
94                     kernel.unroll, sum, expected);
95             return 1;
96         }
97
98         double ns_per_element = (double) min_ns / n / calls;
99         printf("%d,%lld,%.4f,%.3f\n", kernel.unroll, min_ns,
100              ns_per_element,
101              ns_per_element * core_ghz);
102     }
103     free(a);
104     return 0;
105 }

```